

Vision-based Obstacle Tracking and IMU for the Robotic Fish Steering Navigation

Abdul Azhar Jamesh, Deepak Prabhu, Pranesh M, Sandhiya D, Akhil VM
Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, India

Abstract—Navigating autonomously underwater is challenging because the underwater environment introduces inherent difficulties such as turbid water, varying lighting conditions, and water movements that affect both perception and balance. In this paper, we present a system that allows an autonomous robotic fish to navigate using inertial measurements while avoiding obstacles through vision. An adaptive image enhancement system controlled by a high-pressure neural network improves each captured underwater image before the robot determines the environment ahead. After enhancement, the frame is processed using a YOLOv8 detector based on a convolutional neural network to identify nearby obstacles, which are then represented using bounding markers. The marker position is provided to a controller based on the artificial potential field method to determine the steering angle. The body may deviate from its desired direction due to water currents; therefore, deviation information from the MPU6050 inertial sensor is used to provide stabilizing feedback. The final steering angle is transmitted via Wi-Fi to an MG995 servo motor that actuates the robotic fish tail fin. Experimental tests demonstrate successful obstacle avoidance and the ability to maintain heading during hardware-in-the-loop experiments when navigation and inertial sensor data are combined.

Index Terms—obstacle avoidance, yolo-v8, adaptive image enhancement, sensor fusion

I. INTRODUCTION

Underwater robots are widely utilized for environmental monitoring and related applications, but navigation remains challenging [1]. The nonlinear hydrodynamic effects and limited control options make stable motion control a challenging task. The proportional-derivative (PD) controller, one of the most widely used in underwater vehicles, discards high-frequency mechanical oscillations but compensates well for low-frequency heading drift. Vision-inertial odometry is a well-established sensor fusion mechanism used as a state estimator for robotic systems, through the combination of the two visual and inertial modalities [18]. If navigation is done using visual information, the IMUs' orientation information can be used for oscillation correction and drift compensation. The MPU6050 is a compact MEMS 6DOF motion sensor that contains a three-axis accelerometer and three-axis gyroscope. Its size and ability to independently process motion makes it a common sensor for use in embedded robotic systems [3]. YOLOv8 [4] has a better detection accuracy and faster inference speed, features that make it useful for realtime robotic navigation tasks. Robotic fish navigation using vision has been researched and is described in [2], where avoidance steering and formation control are both driven by received image feedback, though vision-only techniques still suffer from drift

due to steering actuation delay and disturbances so inertial drift compensation was proposed in this work.

Our system is constructed atop the discussed fundamentals integrating a single wireless perception-control system merging HPNN based image optimization, YOLOv8 detection, APF based navigation, and IMU stabilization. The theory was verified through HIL tests. The contributions can be summarized as:

- A vision-inertial navigation framework that combines YOLOv8 [4] obstacle detection with MPU6050 based heading stabilization with a feedback closed-loop perception-control that prevents heading drift from occurring as a result of servo actuation and disturbances in the environment.
- A module for adaptive image enhancement that is directed by HPNN (AIE module). The module forecasts regarding each frame's improvement parameters and is trained in tandem with the detection backbone, resulting in low light making objects easier to see, and colour loss depends on wavelength [11] and scattering caused by particles [12], without requiring oversight of the clear image.
- Adaptive gain APF with modulation to avoid hindrances, where repulsion gain value changes by using temporal smoothed obstacles risk parameter formulated from pre-determined bounding box region [5]. It ensures smooth navigation in cluttered scenarios.

The proposed system uses perception–decision–control pipeline for self-driving. The ESP32 camera captures image frames that are sent to a host computer with a GPU for visual enhancement and object detection. The visual visualization enhancing module adapted from the HPNN enhances the visual perception of the target scene prior to obstacle detection using YOLOv8. The positions of detected obstacles are then processed using an APF controller to derive the repulsive steering forces. Meanwhile, inertial readings from the MPU6050 provide heading, which is used to correct drift through a PD controller. The combined steering command is wirelessly sent to the MG995 servo motor which moves the robotic fish tail fin for obstacle avoidance.

II. ROBOTIC FISH BUILDING SYSTEM AND PLATFORM

The navigation framework complies with the perception–decision–control pipeline that is used a lot in robotic fish systems [2], as illustrated in Fig. 1.

A. Configuring the Platform

The hardware platform consists of several components used to construct the robotic fish. An ESP32 microcontroller handles communication, and an MPU6050 is used as an inertial measurement unit to detect the posture, and an MG995 servo to control the tail fin.

The tail fin of this robotic fish is controlled by a MG995 servo motor to make it move forward and backward. It has an onboard MPU6050 with triaxial gyroscope and accelerometer data. The camera module of the ESP32 fetches image frames to do wireless inference through the host.

B. The System's Structure

The captured image frames are transmitted using the available Wifi on the ESP32 camera to a processing unit. The frames are first passed through the AIE module with HPNN prior to detection. This is followed by the YOLOv8 algorithm to detect obstacles present in the scene and return the bounding boxes with the corresponding corner points (x_1, y_1, x_2, y_2) . With this architecture, deep learning inference that is computationally heavy is executed on a different device, eliminating an overhead of components.

Horizontal distance of each obstacle from the image center can be expressed as:

$$d_i = \frac{c_i - W/2}{W/2} \quad (1)$$

where the c_i denotes the center of the obstacle and W is the width of the image. This number denotes the direction and how much of a portion of the steering contribution is to be done.

The tail mechanism of the robotic fish motor controls the navigation commands of MG995 to change the direction of the fish. The final command for steering the fish is the combination of navigation results based on its heading and vision (which includes the IMU correction) and sent as a command of PWM to the actuator through the servo.

III. METHODOLOGY

The implementation of the proposed system was carried out in four steps, which include: dataset designing, training the model, integration of the system and testing.

A. Creating and Preparing the Dataset

The training dataset was collected using the OV2640 camera of 640×480 . The dataset was annotated and divided into a 80:10:10 ratio of train, validation, and test. Data normalisation and augmentation techniques were utilised to improve robustness.

B. Training for the HPNN and AIE Module

Both the HPNN and YOLOv8 detection backbone is end to end trained. A down sampled version of the input frame is supplied to the network and it outputs five scalar parameters. These AIE filter parameters (details in Section IV) allow to control contrast and colour balance strength, and sharpening

intensity to enhance the detector output. All filtering operations are differentiable, and thus loss in detection can be backpropagated through the improver module in the course of training. Therefore, HPNN parameters are to be trained along with the detector. There is no requirement for a distinct clear image ground truth.

C. Configuring and Developing the YOLOv8 Model

The YOLOv8 selection for the backbone detection is due to its impressive accuracy combined with inference speed of identification [4]. At first, the model was initialized with pre-trained weights of ImageNet and then re-trained with demanding dataset.

For this study, the YOLOv8n (nano) variant was selected as it is an ideal lightweight model in terms of accuracy and speed for the embedded deployment. The standard detection metrics mAP₅₀, mAP₅₀₋₉₅, recall, and precision were reported.

D. System Integration

After developing and training each module independently, these modules were connected to create a pipeline having the distributed hardware-software architecture. The image capturing is done by the host through HTTP, the AIE preprocessing is at the end of the trainable HPNN (OpenCV), the YOLOv8 detection (PyTorch), the command generation using the APF and PD correction using IMU data. Finally, the output (steering command) is sent to the ESP32 using PWM.

Whenever the neural network must make a decision, for a few frames it stacks the intermediate frames until there are enough to make a decision, so all intermediate frames reuse the final detection. The decision on drawing conclusions, however, is performed on a background thread to ensure the main loop is not blocked always. Furthermore, the actuator commands are not updated every time; they are updated every 0.12 s, which means the control loop is independent of how long it takes for the neural network to make a decision.

E. Procedure for Assessment

The system was evaluated in three dimensions: in terms of a detection quality (mAP₅₀, mAP₅₀₋₉₅, recall, accuracy, and normalised confusion matrix), navigating quality (servo performance for four cases: no obstacle, left, right, and multiple), and timeliness (latency, control update frequency, and detection processing rate).

IV. VISUAL RECOGNITION SYSTEM

A YOLOv8-based autonomous vision system is used to detect environmental obstacles. To boost the performance of AIE, we propose a dynamic image enhancement module.

A. Identifying Challenges with YOLOv8

Feedback from real-time inference applied to the frames from an ESP32 camera are shared with the host. Boxes of obstacles are defined by their corner coordinates. Based on obstacles spotted around the center of frame image, the robot chooses left, right or go straight in ranges with order from strong to weak reaction. In this way, when center image

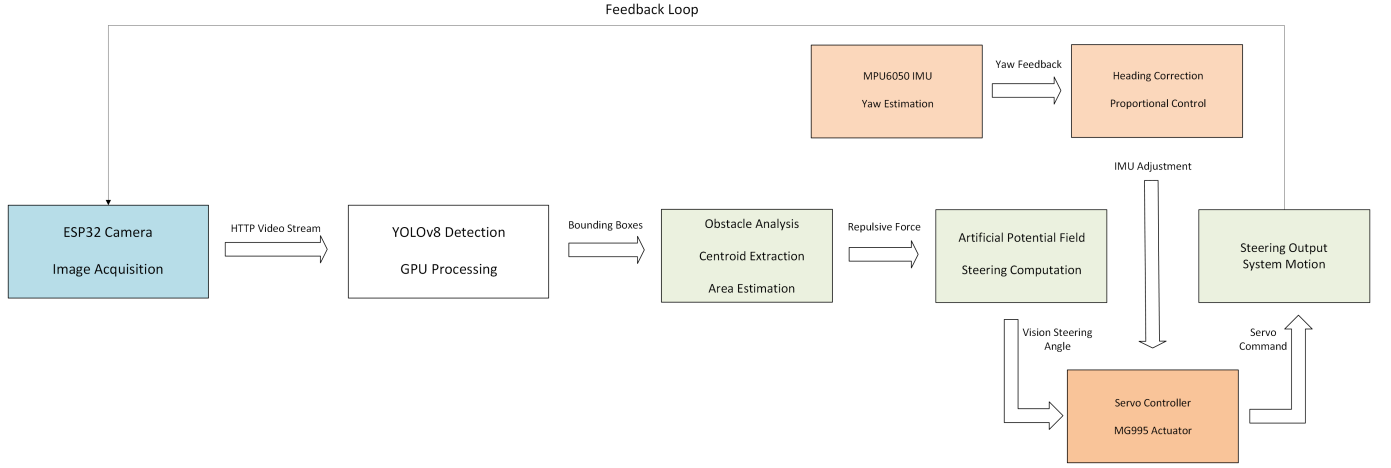


Fig. 1: Proposed system architecture illustrating the HPNN–AIE preprocessing stage, YOLOv8 obstacle detection, APF navigation decision making, IMU-based PD stabilization, and actuator control within a closed-loop perception–control framework.

contains boxes, the robot reacts stronger to turn to avoid obstacle, as crashes are most likely to occur in center position area. The distance to the obstacle is estimated using the bounding box area as this gets larger so does the decision to avoid.

B. Designing the AIE Module

Underwater, the suspended particles create several disturbances for the visible light, like scattering, dimming, and desaturation. All these characteristics enhance the difficulties of the object detection. But before the detection, the AIE module is introduced for increasing the visual perception accuracy. A portable HPNN analyses the visual features of every frame to enhance the sharpness, contrast and color balance parameters.

The improvement of the picture requires the three steps. Channel-wise colour balancing is:

$$I_w = (W_r I_r, W_g I_g, W_b I_b) \quad (2)$$

where I_r , I_g , and I_b denote the RGB channels, and W_r , W_g , W_b are the per-channel weights generated from HPNN. The contrast enhancement is as follows:

$$I_c = \omega \text{En}(I) + (1 - \omega)I \quad (3)$$

Where, ω adjusts the contribution of enhanced image and original image. For sharpening the details where we need to enhance the edges we can:

$$I_s = I + \lambda(I - \text{Gaussian}(I)) \quad (4)$$

The enhanced image was created with sharpness factor λ as an integer number. The prediction of obstacle through YOLOv8 based detector configurations was carried out on the sharpened image I_s . The figure 2 depicts HPNN–AIE architecture's preprocessing pipeline.

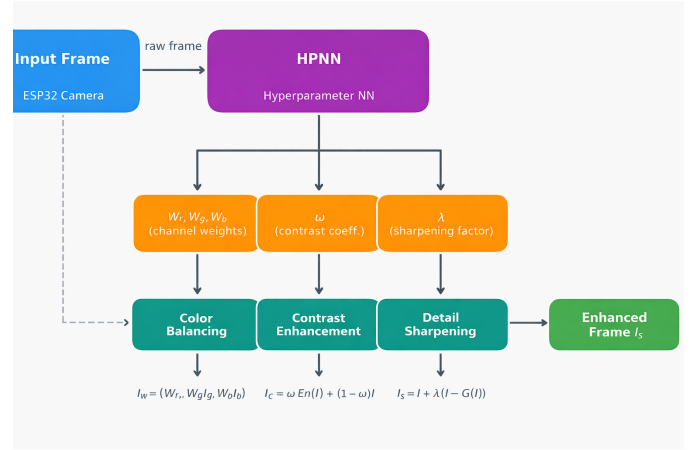


Fig. 2: HPNN-guided adaptive image enhancement pipeline. The HPNN predicts channel weights, contrast coefficient, and sharpening factor from the input frame. The three sequential operations produce an enhanced image passed to the YOLOv8 detector.

C. Calculating the Distance to a Barrier

Depth cannot be directly captured by monocular cameras. The distance is calculated using the normalised bounding box area:

$$A_i = \frac{(x_2 - x_1)(y_2 - y_1)}{HW} \quad (5)$$

Where H and W correspond to the height and width of the image, respectively. In the case of perspective projection:

$$A_i \propto \frac{1}{D_i^2} \quad (6)$$

where D_i is the distance to the obstacle. The bigger the bounding box, the more obstacles are present in a certain area, which reinforces avoidance behavior. Then, a step of spatial

filtering allows to remove the redundant bounding boxes, so that the controllers are not confused by multiple detections. Indeed, if the distance between the centers of two boxes is horizontal is less than 50 pixels, they are considered as duplicates and only one representative will be kept.

V. NAVIGATION AND CONTROL DESIGN

A. Artificial Potential Fields Navigation

Models for avoidance of obstacles based on an artificial potential field (APF). The steering force is repellent for every obstacle detected. To avoid interaction with far-off objects, a proximity threshold is provided:

$$F_i = \begin{cases} -k_r d_i (A_i - A_{th}) & A_i > A_{th} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where k_r is the repulsion gain, d_i is the normalised horizontal displacement, A_i is the normalised bounding box area, and A_{th} is the closeness threshold. The steering force computes for all the known obstacles, and is temporally smoothed to reduce spurious changes due to detection noise:

$$F_s = \frac{1}{N} \sum_{k=1}^N F_k \quad (8)$$

where N is the size of the smoothing window. The lifted risk metric $\bar{\rho}(t)$, representing the history of both screwing and banging risks, is the normalized bounding box area of the last frames (Section V-D), and it influences the adaptive gain as follows:

$$G_s(t) = G_0 (1 + \bar{\rho}(t)) \quad (9)$$

where G_0 denotes the gain at base. This allows to reduce the aggressiveness of steering but maintains smoothness in presence of multiple obstacles if it is clear. The force smoothed is transformed into the vision steering command as $\theta_v = \theta_0 + G_s F_s$ where θ_0 is the servo neutral angle.

B. Heading Stabilisation Using IMU

Heading deviation is created by environmental disturbances and servo actuation. The IMU [3] outputs yaw angle $\psi(t)$, the reference heading $\psi_r = \psi(0)$ is kept when the system powered. The PD control and heading error are:

$$e_\psi = \psi_r - \psi(t), \quad C_{imu} = K_p e_\psi + K_d \dot{e}_\psi \quad (10)$$

gains are thus $K_p = 0.6$ and $K_d = 0.25$ from empirical adjustment.

C. Combination and Control Smoothing

The steering command $\theta_v(t)$ comes from the vision module and the corrections $C_{imu}(t)$ from the IMU, all together are summed to compute the intermediate steering signal $\theta(t)$:

$$\theta(t) = \theta_v(t) + C_{imu}(t) \quad (11)$$

$$\theta_s(t) = \alpha \theta(t) + (1 - \alpha) \theta_s(t-1) \quad (12)$$

where parameter α is smoothing factor, which indicates how fast the steering response reacts to new inputs.

D. Navigation Algorithm

A pseudo-code for the navigation logic implemented in the robotic fish control is listed in Algorithm 1. It combines inertial controls with visual obstacles detection to provide smooth steering commands. The increased flexibility that the smoothed danger level creates is:

$$\bar{\rho}(t) = \frac{1}{M} \sum_{k=t-M}^t \sum_i A_i^{(k)} \quad (13)$$

where $A_i^{(k)}$ is the area of the bounding box associated to the i -th obstacle at frame k , and the detections proximity thresholded their contribution to the gain driving G_s in Eq. (9).

Algorithm 1 Hybrid Vision-IMU Navigation Control

- 1: Initialize camera and IMU (UDP): store $\psi_r \leftarrow \psi(0)$
 - 2: **while** system is working **do**
 - 3: Obtain frame; apply HPNN-guided AIE
 - 4: YOLOv8 detection (each N frames); duplicate extract
 - 5: Context: A_i (Eq. 5), d_i (Eq. 1), $\bar{\rho}$ (Eq. 13)
 - 6: APF gating $\Rightarrow F_i$ (Eq. 7); smooth $\Rightarrow F_s$ (Eq. 8)
 - 7: $G_s = G_0(1 + \bar{\rho})$; $\theta_v = \theta_0 + G_s F_s$
 - 8: Read $\psi(t)$; PD correction $\Rightarrow C_{imu}$ (Eq. 10)
 - 9: Fuse and smooth: θ_s (Eqs. 11-12)
 - 10: Send θ_s to MG995 servo
 - 11: **end while**
-

TABLE I: Control and Navigation Parameters

Parameter	Value
Yaw proportional gain K_p	0.6
Yaw derivative gain K_d	0.25
Base steering gain G_0	45
Repulsion gain k_r	8
Proximity threshold A_{th}	0.12
Steering smoothing factor α	0.35
Actuator update interval	0.12 s

VI. EXPERIMENTAL VALIDATION

Indoor experiments were conducted to evaluate detection accuracy, navigation stability, and steering performance using artificial barriers.

The experiments show that the trained YOLOv8 model achieves a mean average precision (mAP) of 0.733 at IoU=0.50 and 0.456 at IoU=0.50:0.95. The accuracy of the created model is in the vicinity of 0.78, and its recall is 0.71. The results indicate reliable detection across all object classes.

Initial tests showed that the heading drift would increase significantly whenever the steering was made entirely based on vision. Feedback from the inertial measurement unit (IMU) was really helpful in correcting the problems that the robot experienced in trying to hold its heading angle. These results highlight the benefits of combining visual perception with inertial feedback.

The steering responses and run-time observation is shown in Table II. Positive APF force results in right steering,

while negative force results in left motion. In free space, the servo returns to neutral. The test with vision-only control demonstrated the servo motion to be less oscillatory because of the IMU-based stabilisation.

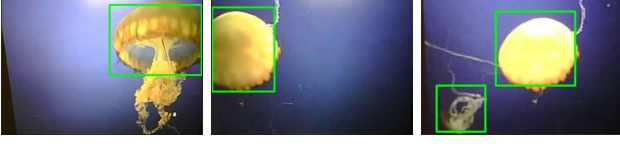


Fig. 3: Experimental obstacle avoidance results. (a) Right-side obstacle producing left steering, (b) left-side obstacle producing right avoidance, (c) multiple obstacle scenario with adaptive steering.

TABLE II: Experimental Results: Steering Responses and System Performance

Obstacle Avoidance Reaction			
Scenario	APF Force	Steering (°)	Motion
No Obstacle	0.00	88–92	Forward
Left Barrier	+0.40–0.60	105–118	Right Avoid
Right Obstacle	−0.40–0.60	60–70	Left Avoid
Multiple Obstacles	±0.30	85–108	Adaptive
Runtime Performance			
Detection Accuracy (mAP ₅₀)	0.733		
Processing Rate	~5.0 FPS		
Control Update Frequency	~2.1 Hz		
System Latency	380–500 ms		

VII. CONCLUSION AND FUTURE WORK

We developed and validated experimentally the vision-inertial based robotic fish navigation system adapted for obstacle avoidance. The system consisted of YOLOv8 with embedded HPNN-adapted appearance learning for objects identification and APF for the navigation system. PD based inertial control has also been embedded to accommodate the heading control requirement. The system successfully demonstrated the object detection and provided with consistent avoidance actions between the trials. Most interestingly, the IMU based heading stabilisation has significantly stabilised the servos' oscillation behaviour that was prominent under pure vision control, thus providing conclusive evidence the successful fusing of the vision and inertial based IMU sensors system.

Future development will focus on fully autonomous marine exploration with integrated propulsion control, conducting trials in realistic open water environments and handling dynamic and unstructured sub-surface environments utilizing reinforcement learning for driving.

REFERENCES

[1] A. Prakash, A. R. Nair, H. Arunav, R. P. Rthuraj, V. M. Akhil, C. Tawk, and K. V. Shankar, "Bioinspiration and biomimetics in marine robotics: A review on current applications and future trends," *Bioinspiration & Biomimetics*, vol. 19, no. 3, Art. no. 031002, Apr. 2024.

[2] J. Han, R. Huang, X. Yuan, Y. Liu, B. Yin, S. Zou, and Z. Ma, "Vision-Based Obstacle Avoidance and Formation Control for Underwater Robotic Fish," *IEEE Robot. Autom. Lett.*, vol. 10, no. 10, pp. 10442–10449, 2025, doi: 10.1109/LRA.2025.3599014.

[3] H. Jian, "Angle Detection System Design Based on MPU6050," in *Proc. Int. Conf. Elect. Eng. Mechatron. Control (ICEMC)*, 2017, doi: 10.2991/icemc-17.2017.2.

[4] R. Varghese and M. Sambath, "YOLOv8: An Innovative Object Detection Algorithm with Improved Robustness and Performance," in *Proc. Int. Conf. Adv. Data Eng. Intell. Comput. Syst. (ADICS)*, 2024, pp. 1–6, doi: 10.1109/ADICS58448.2024.10533619.

[5] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 1985, vol. 2, pp. 500–505, doi: 10.1109/ROBOT.1985.1087247.

[6] S. Liu, C. Liu, G. Wei, L. Ren, and L. Ren, "Design, Modeling, and Optimization of Hydraulically Powered Double-Joint Soft Robotic Fish," *IEEE Trans. Robot.*, vol. 41, pp. 1211–1223, 2025, doi: 10.1109/TRO.2025.3526087.

[7] S. Li, Z. Wu, S. Dai, J. Wang, M. Tan, and J. Yu, "Tight-Space Maneuvering of a Hybrid-Driven Robotic Fish Using Backstepping-Based Adaptive Control," *IEEE/ASME Trans. Mechatron.*, vol. 29, no. 5, pp. 3219–3231, 2024, doi: 10.1109/TMECH.2023.3338879.

[8] M. Sfakiotakis, D. M. Lane, and J. B. C. Davies, "Review of Fish Swimming Modes for Aquatic Locomotion," *IEEE J. Ocean. Eng.*, vol. 24, no. 2, pp. 237–252, 1999, doi: 10.1109/48.757275.

[9] X. Cong, Y. Zhao, J. Gui, J. Hou, and D. Tao, "A Comprehensive Survey on Underwater Image Enhancement Based on Deep Learning," *arXiv:2405.19684*, 2026.

[10] C. Qiu, Z. Wu, J. Wang, M. Tan, and J. Yu, "Autonomous Recovery Control of Biomimetic Robotic Fish Based on Multi-Sensory System," *IEEE Robot. Autom. Lett.*, vol. 9, no. 2, pp. 1420–1427, 2024, doi: 10.1109/LRA.2023.3346813.

[11] C.-Y. Li, J.-C. Guo, R.-M. Cong, Y.-W. Pang, and B. Wang, "Underwater Image Enhancement by Dehazing With Minimum Information Loss and Histogram Distribution Prior," *IEEE Trans. Image Process.*, vol. 25, no. 12, pp. 5664–5677, 2016, doi: 10.1109/TIP.2016.2612882.

[12] D. Akkaynak and T. Treibitz, "A Revised Underwater Image Formation Model," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 6723–6732, doi: 10.1109/CVPR.2018.00703.

[13] J. Yu, Z. Wu, X. Yang, Y. Yang, and P. Zhang, "Underwater Target Tracking Control of an Untethered Robotic Fish With a Camera Stabilizer," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 10, pp. 6523–6534, 2021, doi: 10.1109/TSMC.2019.2963246.

[14] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[15] P. Yao, X. Sui, Y. Liu, and Z. Zhao, "Vision-Based Environment Perception and Autonomous Obstacle Avoidance for Unmanned Underwater Vehicle," *Appl. Ocean Res.*, vol. 134, Art. no. 103510, 2023, doi: 10.1016/j.apor.2023.103510.

[16] S. Fayaz, S. A. Parah, G. J. Qureshi, J. Lloret, J. Del Ser, and K. Muhammad, "Intelligent Underwater Object Detection and Image Restoration for Autonomous Underwater Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 1726–1735, 2024, doi: 10.1109/TVT.2023.3318629.

[17] J. Wang and X. Tan, "Averaging Tail-Actuated Robotic Fish Dynamics Through Force and Moment Scaling," *IEEE Trans. Robot.*, vol. 31, no. 4, pp. 906–917, 2015, doi: 10.1109/TRO.2015.2433539.

[18] T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, 2018, doi: 10.1109/TRO.2018.2853729.

[19] S. L. Sankeerthi, S. Srinivasan, M. Nohitha, N. Chandana, and V. M. Akhil, "Enhancing Robot Navigation in ROS 2 with Theta* Planning," in *Proc. Int. Conf. Comput. Technol. Data Commun. (ICCTDC)*, 2025, pp. 1–6, doi: 10.1109/ICCTDC64446.2025.11159061.